

Instituto Tecnológico de Costa Rica
Escuela de Ingeniería en Computadores
Paradigmas de Programación (CE1106)
Primer Semestre 2026

Documentación Técnica
Tarea 4: spaCEinvaders

Profesor:
Luis Alonso Barboza Artavia

Estudiantes:

- Frander Picado Mena 2022101904
- Samuel Morales Cascante 2024130487
- Dylan Elizondo Picado 2023086320

Enlace del Repositorio (GitHub):
<https://github.com/Andy0122/spaCEinvaders-CE1106.git>

Fecha de Entrega:
11 de junio de 2026

Contenido

1.1. Descripción de las reglas implementadas.	3
1.2. Descripción de las estructuras de datos desarrolladas	3
1.3. Descripción detallada del o los algoritmos desarrollados.....	5
1.4. Problemas conocidos sin solución	10
1.5. Plan de Actividades realizadas por estudiante	11
1.6. Conclusiones.....	12
1.7. Recomendaciones	13
1.8. Bibliografía.....	14

1.1. Descripción de las reglas implementadas.

El proyecto *spaCEinvaders* consiste en el desarrollo de un videojuego multijugador distribuido basado en el clásico arcade de disparos, implementando una arquitectura Cliente-Servidor. El objetivo principal fue construir una aplicación que combine múltiples tecnologías y paradigmas de programación: el cliente fue desarrollado en C (paradigma imperativo) para garantizar un renderizado gráfico eficiente y la lectura nativa de hardware, mientras que el servidor fue construido en Java (paradigma orientado a objetos) para aprovechar sus robustas capacidades de concurrencia y diseño modular.

El sistema sigue el modelo de **Autoridad del Servidor**, lo que significa que el backend en Java es el único responsable de ejecutar la totalidad de la lógica del juego (colisiones, daño, puntajes, oleadas), mientras que el cliente C se limita a capturar los comandos físicos (mediante un microcontrolador ESP32 conectado por puerto serial) y a dibujar en pantalla los estados recibidos a través de sockets TCP.

1.2. Descripción de las estructuras de datos desarrolladas.

Para satisfacer los requerimientos de rendimiento y comunicación, el proyecto hace uso de distintas estructuras de datos divididas según su entorno (cliente o servidor).

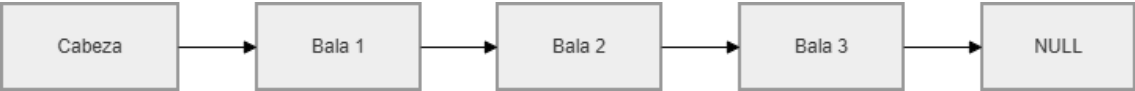
Estructuras utilizadas en el Cliente (C)

El cliente emplea structs de C para almacenar únicamente el estado visual de las entidades gráficas. Las estructuras principales son:

- **Jugador:** Almacena el `id_jugador`, la `posicion_x` del cañón, las vidas restantes y la puntuación acumulada.
- **Extraterrestre:** Define cada enemigo visible mediante su `id`, coordenadas `x` e `y`, el tipo (calamar, cangrejo o pulpo) y su estado.
- **Búnker:** Modela los escudos defensivos guardando su `id`, coordenadas y el porcentaje `salud`, valor que el cliente utiliza para cambiar la textura según el nivel de daño.
- **OVNI:** Representa la nave especial mediante su `id`, coordenadas, velocidad, puntosExtra asociados y una bandera activo.

Para administrar los proyectiles visibles sin agotar la memoria, se implementó desde cero una **Lista Simplemente Enlazada**. Esta estructura permite una inserción $O(1)$ y un uso de memoria dinámico según la cantidad de disparos presentes en pantalla. Su funcionamiento base consiste en una

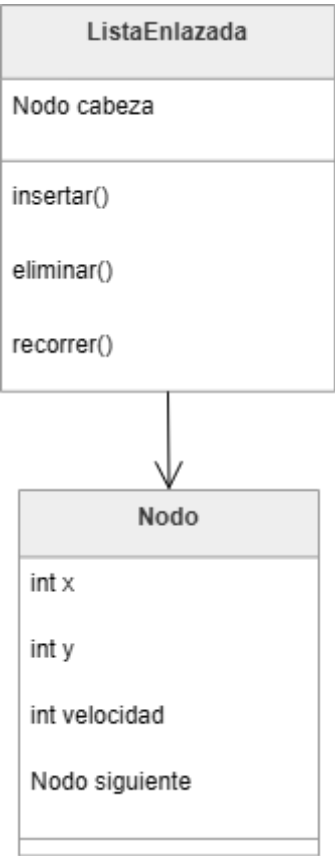
estructura ListaBala (con una referencia a la cabeza y la cantidad total) y una estructura NodoBala (que guarda las coordenadas, la velocidad y un puntero al siguiente nodo).



Estructuras utilizadas en el Servidor (Java)

Dado que el servidor administra la lógica oficial y atiende múltiples conexiones simultáneas, se priorizó el uso de colecciones seguras para subprocesos (Thread-safe) y estructuras de acceso rápido.

Al igual que en el cliente, las balas activas se almacenan utilizando una implementación propia de una lista simplemente enlazada orientada a objetos en Java, facilitando la creación y eliminación constante de proyectiles en cada "tick" de actualización.



Para la gestión de entidades masivas como los **alienígenas y el OVNI**, se utilizaron diccionarios `ConcurrentHashMap<Integer, Extraterrestre>`. Esta elección permite búsquedas mediante ID en tiempo $O(1)$ y garantiza que las actualizaciones concurrentes desde los distintos hilos de red no generen excepciones. Por otro lado, los **búnkeres** se guardan en

un `ArrayList<Bunker>` sincronizado, ya que su tamaño es fijo (4 escudos) y su acceso secuencial es muy frecuente durante la verificación de colisiones.

Finalmente, el servidor soporta **múltiples partidas simultáneas**. Para lograrlo, la clase `GestorPartidas` emplea otro `ConcurrentHashMap` que asocia cada identificador de sala con un objeto `Juego` independiente, el cual encapsula todas las listas y mapas de esa partida específica.

1.3. Descripción detallada de los algoritmos desarrollados

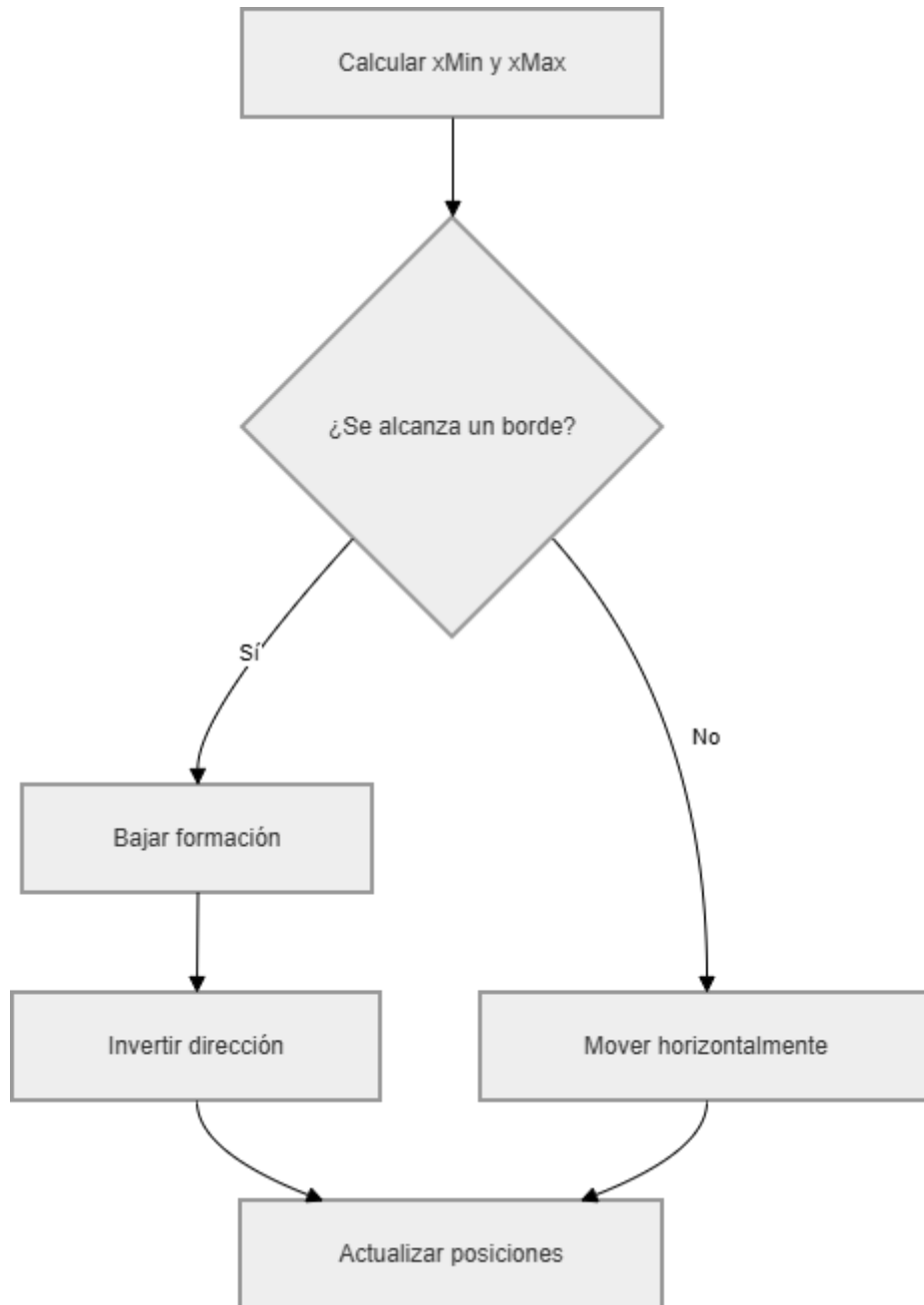
A continuación, se detallan los algoritmos fundamentales que controlan las mecánicas del juego:

Ciclo principal del juego: El núcleo de la aplicación reside en el servidor mediante un `ScheduledExecutorService` que ejecuta un "tick" cada 200 milisegundos. En este intervalo, el servidor recalcula todo: mueve los alienígenas y OVNIs, genera disparos enemigos aleatorios, actualiza la posición de todas las balas, verifica las colisiones, actualiza las puntuaciones de los jugadores y evalúa si se alcanzó una condición de victoria o derrota.



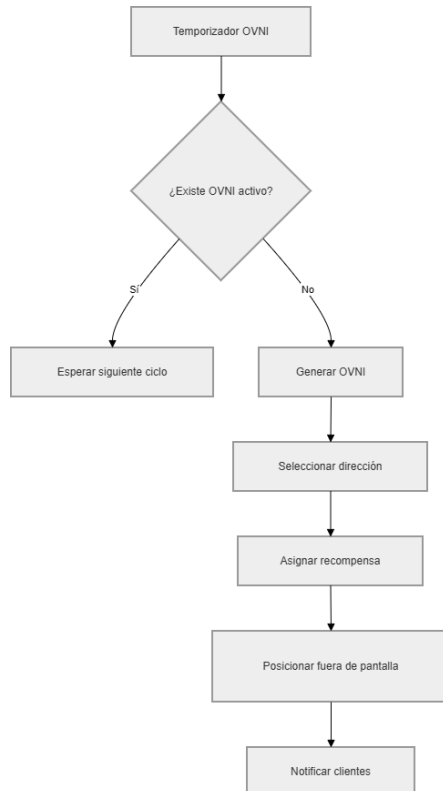
1.3.3 Algoritmo de movimiento de alienígenas

Algoritmo de movimiento de alienígenas: Este algoritmo calcula continuamente los límites extremos izquierdo y derecho de la formación invasora utilizando las posiciones de todos los alienígenas activos. Si la formación toca uno de los bordes de la pantalla, el algoritmo ordena que todos los extraterrestres desciendan una fila y que inviertan su dirección horizontal. Si no tocan los bordes, simplemente continúan desplazándose de forma lateral.



Algoritmo de aparición del OVNI: Mediante un proceso independiente al ciclo principal, el servidor evalúa cada cierto intervalo de tiempo si existe un OVNI activo. Si no hay ninguno, el algoritmo crea uno asignándole aleatoriamente una dirección de movimiento y un puntaje de recompensa. Luego, lo posiciona fuera

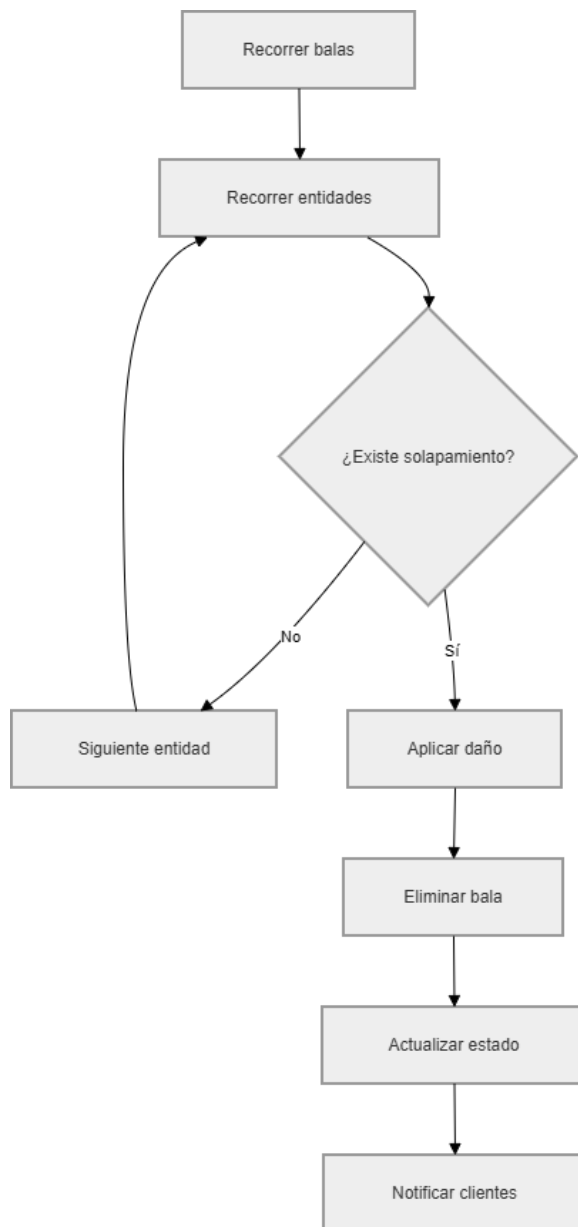
de los límites de la pantalla y notifica su aparición a los clientes.



Algoritmo de disparo: Cuando el servidor recibe la solicitud de disparo de un jugador, valida el comando, crea una nueva bala y la inserta en la lista enlazada. De forma paralela, en cada ciclo, el servidor evalúa una probabilidad aleatoria para cada alienígena activo; si se cumple la condición matemática, ese alienígena en particular genera una bala enemiga que también es agregada a la lista de proyectiles en descenso.

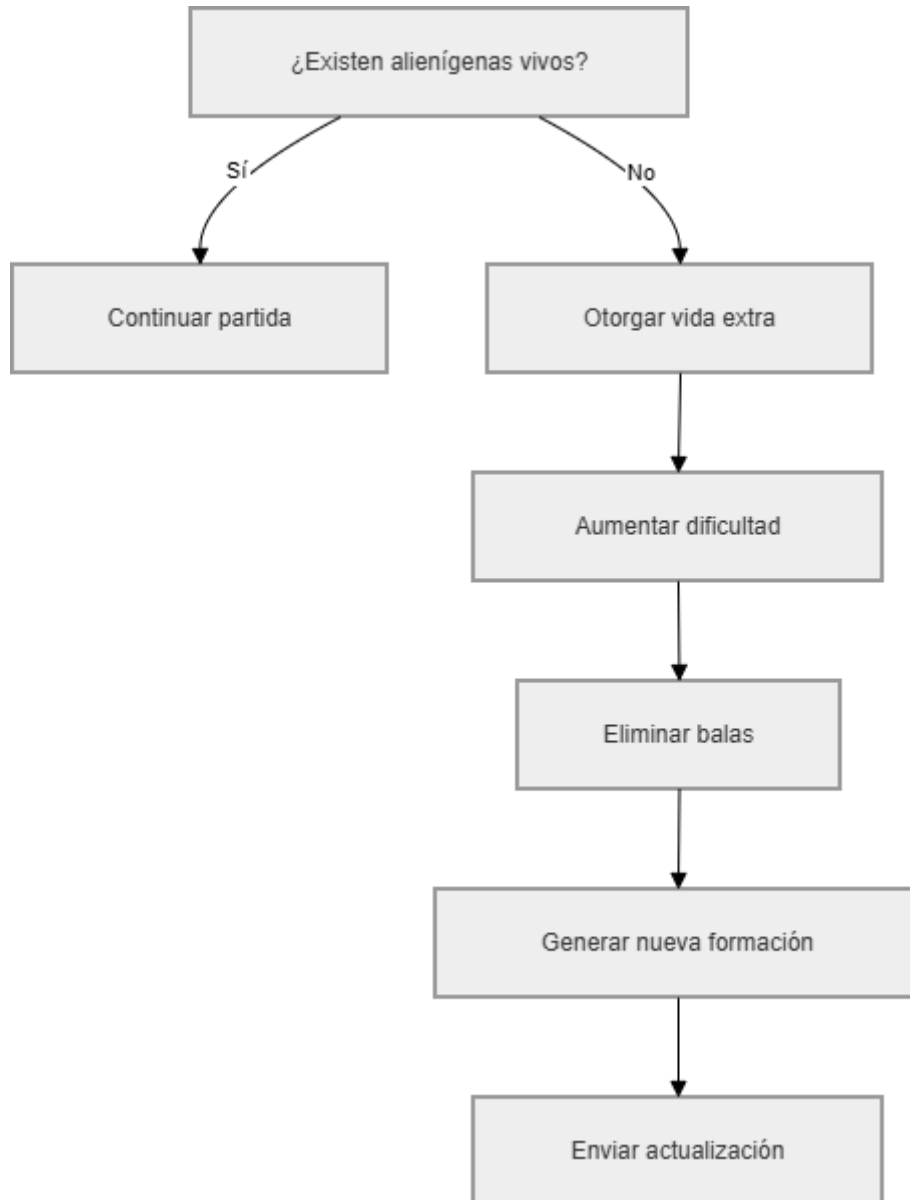


Algoritmo de detección de colisiones: Se ejecuta exclusivamente en el servidor utilizando rectángulos de colisión (Axis-Aligned Bounding Box o AABB). Itera secuencialmente sobre la lista de balas cruzando sus coordenadas con los hitboxes de alienígenas, OVNI, búnkeres y jugadores. Al detectar un solapamiento matemático, aplica el daño correspondiente, elimina la bala, marca la entidad impactada para su destrucción y suma los puntos al jugador.

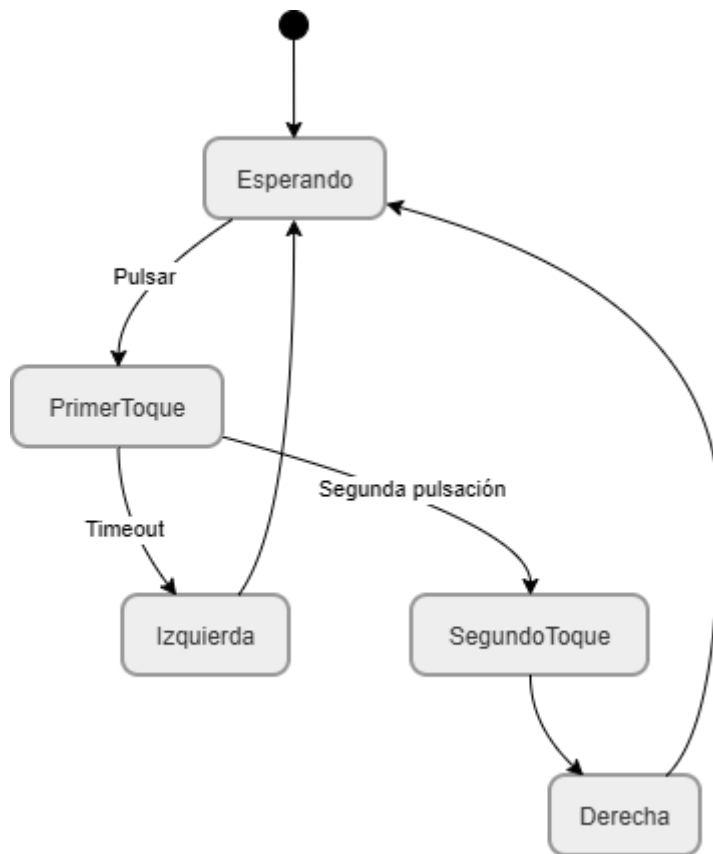


Algoritmo de reinicio de oleadas: Cuando el servidor detecta que el mapa de extraterrestres está vacío, activa la rutina de reinicio: otorga una vida extra al jugador, aumenta la dificultad incrementando la velocidad de avance, elimina todos los proyectiles en pantalla, genera una nueva formación alienígena y

distribuye el comando de limpieza y actualización a los clientes.



Algoritmo de control físico (ESP32): El microcontrolador ejecuta una máquina de estados para interpretar el uso de los dos botones físicos. El algoritmo mide el tiempo de pulsación: un toque simple en el botón principal se lee como desplazamiento a la izquierda, una doble pulsación como desplazamiento a la derecha, y presionar el botón secundario activa el disparo.



1.4. Problemas sin solución

Aunque se alcanzaron todos los objetivos funcionales, se detectaron limitaciones operativas que permanecen en el sistema:

1. **Reinicio automático de partidas:** Actualmente, el servidor conserva el estado de cada sala mientras permanezca en ejecución. Si todos los jugadores abandonan una partida, esta se pausa para no consumir recursos (la iteración se detiene), pero los datos quedan almacenados en memoria. Como consecuencia, si los jugadores se reconectan más tarde a esa sala, la partida continúa exactamente donde la dejaron, incluso si previamente había terminado en un estado definitivo de *Game Over*. Esto exige un reinicio manual del servidor para depurar las salas finalizadas.
2. **Robustez frente a mensajes malformados:** El protocolo implementado asume que los clientes envían mensajes correctamente estructurados (separados por el delimitador |). Aunque se diseñó un mecanismo para lidiar con la fragmentación de la red, no existe una validación exhaustiva o un esquema de autenticación contra mensajes deliberadamente manipulados, lo que en entornos no académicos podría desencadenar comportamientos inesperados o excepciones en el servidor.

1.5. Plan de Actividades realizadas por estudiante

ID	Descripción de la Actividad	Tiempo Estimado	Responsable a Cargo	Fecha de Entrega
A1	Arquitectura Base y Red: Esqueleto de carpetas, Makefile, configuración de Sockets TCP y Handshake inicial.	4 horas	Dylan	15/Mayo
A2	Lógica del Servidor (Modelo y Patrones): Creación de clases en Java (Jugador, Alien, Bunker). Implementación de 2 patrones de diseño (ej. <i>Factory</i> para crear aliens, <i>Observer</i> para notificar a clientes).	8 horas	Frander	20/Mayo
A3	Interfaz Gráfica (Cliente C): Inicialización de ventana gráfica (ej. Raylib/SDL). Dibujo del cañón, matriz de aliens y bunkers. Separación estricta de constantes en constantes.h.	10 horas	Samuel	23/Mayo
A4	Controlador Raspberry Pi Pico: Programación en C del microcontrolador. Lógica de botones (1 toque = Izq, 2 toques = Der, Botón 2 = Disparo) y envío de señales por puerto UART/I2C.	8 horas	Dylan	25/Mayo
A5	Parseo y Serialización (Integración Red): Envío de coordenadas desde Java y actualización de structs en C. El cliente debe poder leer comandos tipo "Crear Extraterrestre (X,Y,Pts)" y dibujarlos.	8 horas	Samuel (Parte en C)	28/Mayo
A6	Motor del Juego (Reglas en Java): Detección de colisiones, control de 3 vidas, suma de puntuación (10, 20, 40 pts), aparición aleatoria del OVNI y aumento de velocidad al limpiar la pantalla.	10 horas	Frander	01/Junio
A7	Integración Hardware-Juego: Lectura del puerto serial (UART) en el cliente C para que la Raspberry Pi Pico mueva el cañón y dispare en la interfaz gráfica.	6 horas	Dylan	3/Junio

A8	Cliente Espectador y Multijugador: Validar soporte mínimo de 2 jugadores. Habilitar rol "ESPECTADOR" que solo reciba coordenadas y dibuje, sin poder enviar comandos.	5 horas	Samuel (Parte en C)	5/Junio
A9	Documentación Técnica y Manual: Redacción de estructuras de datos, algoritmos, problemas sin solución y manual de usuario.	6 horas	Todos	8/Junio
A10	Testing y Defensa: Pruebas integrales de estrés, depuración de código, generación de ejecutables finales y preparación de la defensa.	5 horas	Todos	10/Junio

1.6. Problemas encontrados

Durante el desarrollo de esta arquitectura distribuida surgieron diversos contratiempos que requirieron iteraciones de diseño:

1. Desincronización de colisiones entre cliente y servidor:

En etapas iniciales, el cliente calculaba parcialmente los impactos de las balas. Esto provocaba que, en el multijugador, un cliente pudiera ver a un enemigo destruido mientras otro cliente, por latencia, aún lo veía vivo. La solución consistió en aplicar la Autoridad del Servidor de forma estricta: se trasladó absolutamente toda la lógica matemática de colisión a Java, obligando a los clientes a depender únicamente de las órdenes de destrucción enviadas por el servidor.

2. Alienígenas "fantasma" durante el cambio de oleada:

Al completar una oleada, el servidor mandaba la nueva formación enemiga, pero algunos clientes mantenían en su memoria local (structs) a los enemigos del nivel anterior, mezclando las oleadas visualmente. Para erradicar estos entes "fantasma", se diseñó la instrucción de red LIMPIAR_ALIENS, la cual el servidor dispara justo antes de enviar la nueva oleada para forzar al cliente a purgar su memoria gráfica.

3. Desfase visual (Tunneling) en los proyectiles:

El motor gráfico del cliente renderiza a 60 FPS, mientras que el servidor ejecuta la lógica cada 200 ms. Esta gran brecha temporal ocasionaba que visualmente la bala atravesara al enemigo de un lado a otro entre dos actualizaciones de servidor, fallando la colisión aparente. Para solventarlo sin sobrecargar la red, el cliente realiza una interpolación del avance de la bala, mientras que el servidor amplió el rango de detección (hitbox compensatorio) mediante un barrido vertical que evita que los objetos atravesen las colisiones matemáticamente.

4. Fragmentación de mensajes TCP en red:

Durante las pruebas de estrés, comandos combinados llegaban partidos (por ejemplo, ALIEN|15| en un paquete y 20|150 en otro). Como resultado, la función de parseo de C fallaba al encontrar datos incompletos. Esto se resolvió diseñando un acumulador (buffer) de fragmentos en el cliente; el sistema ahora lee el socket, lo almacena temporalmente y únicamente procesa la instrucción cuando detecta el salto de línea (\n) que garantiza que el mensaje está intacto.

5. Zombificación de recursos:

Las desconexiones forzosas de clientes dejaban sockets y ciclos de partidas ejecutándose "al vacío", consumiendo memoria y CPU. Se incorporó una rutina de control que captura la pérdida de conexión del cliente, cierra la referencia del socket y suspende (pausa) la ejecución del ciclo de esa partida de forma inmediata para mantener limpio el entorno del GestorPartidas.

1.7. Conclusiones del proyecto

El desarrollo de *spaCEInvaders* consolidó los conocimientos sobre arquitecturas Cliente-Servidor. Se demostró de manera práctica que centralizar la toma de decisiones en el servidor (Autoridad del Servidor) es el único método verdaderamente efectivo para eliminar inconsistencias y garantizar un entorno justo y uniforme para todos los participantes.

Asimismo, la coexistencia de diferentes paradigmas probó ser altamente efectiva: la programación orientada a objetos (Java) facilitó inmensamente la gestión ordenada de hilos paralelos y estados simultáneos (salas), mientras que el paradigma imperativo estructurado (C) otorgó el control directo sobre la memoria local (mediante las listas simplemente enlazadas) y permitió el acceso nativo a puertos de bajo nivel, lo que hizo posible acoplar un microcontrolador físico (ESP32) para gobernar las acciones de un videojuego en línea en tiempo real.

1.8. Recomendaciones

Técnicamente, se recomienda implementar como trabajo futuro una recolección de basura de salas inactivas (implementando *timeouts*). De esta forma, si una sala finalizada se abandona por un tiempo predeterminado, el servidor la reiniciará a su estado cero por sí solo.

A nivel de red, la dependencia actual de comandos en texto plano (cadenas con delimitadores |) podría escalar hacia protocolos de serialización binaria en futuras iteraciones. Esto reduciría la saturación del flujo de datos en la red, optimizaría el consumo de procesamiento que actualmente gasta C al hacer *parseo* de strings, y

añadiría una capa implícita de seguridad frente a errores de formato. Finalmente, se sugiere implementar el patrón de diseño *Adapter* en el módulo de controles de C, lo que brindaría la flexibilidad necesaria para intercalar entradas (teclado, ratón o el actual ESP32) de manera transparente y sin alterar la lógica.

1.9. Bibliografía

- **Arduino Documentation. (2025).** *Arduino and ESP32 Development Documentation.*
- **Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994).** *Design Patterns: Elements of Reusable Object-Oriented Software.* Addison-Wesley.
- **Kurose, J. F., & Ross, K. W. (2021).** *Computer Networking: A Top-Down Approach (8th ed.).* Pearson.
- **Microsoft Winsock Documentation. (2025).** *Windows Sockets 2 Documentation.*
- **Oracle Java Documentation. (2025).** *Java Platform, Standard Edition Documentation.*
- **Oracle ScheduledExecutorService / ConcurrentHashMap Documentation. (2025).** *Java Concurrency Reference.*
- **Pacxon. (7 de Mayo de 2026).** *Space Invaders.* Obtenido de Pacxon: <http://www.pacxon4u.com/space-invaders/>
- **Raylib. (2025).** *Raylib Cheat Sheet.* Recuperado de Raylib Official Website.
- **Wikipedia. (7 de Mayo de 2026).** *Space Invaders.* Obtenido de Wikipedia: https://es.wikipedia.org/wiki/Space_Invaders